

# Practically Groovy: Smooth operators

## Overload operators on the Java platform? Groovy says yes!

Skill Level: Intermediate

[Andrew Glover](#) ([andrew@thirstyhead.com](mailto:andrew@thirstyhead.com))

Co-Founder

ThirstyHead.com

[Scott Davis](#) ([scott@thirstyhead.com](mailto:scott@thirstyhead.com))

Founder

ThirstyHead.com

25 Oct 2005

The Java™ language has banned operator overloading, but upstart Groovy says "bring it on!" Find out what you've been missing all these years, as Andrew Glover walks you through everyday uses of Groovy's three categories of overloadable operators in this final, regular installment of *Practically Groovy*.

Many developers who started out using C++ will admit some nostalgia for overload operators such as + and -. Convenient though they are, the polymorphic nature of overridden operators can cause confusion, so operator overloading has been banned from the Java language. The upside of this limitation is clarity: Java developers never have to wonder if the + on two objects adds them together or appends one object to another. The downside is the loss of a valuable shorthand.

### About this series

The key to incorporating any tool into your development practice is knowing when to use it and when to leave it in the box. Scripting (or *dynamic*) languages can be an extremely powerful addition to your toolkit, but only when applied properly to appropriate scenarios. To that end, *Practically Groovy* is a series of articles dedicated to exploring the practical uses of Groovy, and teaching you when and how to apply them successfully.

Well, count on *laissez-faire* Groovy to bring back the shortcut! In this installment of *Practically Groovy*, I introduce you to Groovy's support for *operator ad-hoc polymorphism*, also known as operator overloading. As any C++ developer will tell you, this stuff is both handy and fun, although it should be approached with a healthy dose of caution.

## Three kinds of smooth

I categorize Groovy's overloadable operators into three logical groups: comparison operators, math-like operators, and array-like operators. These groups only capture a subset of the available operators in normal Java programming. For example, logical operators like `&` and `^` are not currently available in Groovy.

Table 1 shows the three categories of Groovy's available overloadable operators:

**Table 1. Groovy's overloadable operators**

|    |                                                                                                            |
|----|------------------------------------------------------------------------------------------------------------|
| 1. | Comparison operators map to the normal Java <code>equals</code> and <code>compareTo</code> implementations |
| 2. | Java math-like operators like <code>+</code> , <code>-</code> , and <code>*</code>                         |
| 3. | The array access-like operator <code>[]</code>                                                             |

## Comparison operators

Comparison operators map to the `equals` and `compareTo` implementations found in the Java language and are typically used as shorthand for sorting in collections.

Table 2 shows Groovy's two comparison operators:

**Table 2. Comparison operators**

| Operator                   | Method                      |
|----------------------------|-----------------------------|
| <code>a == b</code>        | <code>a.equals(b)</code>    |
| <code>a &lt;=&gt; b</code> | <code>a.compareTo(b)</code> |

The operator `==` is shorthand for object equivalence in the Java language, but not for object reference identity. In other words, Groovy's `==` placed between two objects means that they're the same because their properties are equal, although each object points to a separate reference.

According to the Javadoc, the Java language's `compareTo()` method returns a negative integer, zero, or a positive integer as this object is less than, equal to, or greater than the specified object. Because this method can return one of three values, Groovy has augmented the `<=>` syntax with the four additional values shown

in Table 3:

**Table 3. Four additional values**

| Operator               | Meaning                                                                                                                        |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <code>a &gt; b</code>  | if <code>a.compareTo(b)</code> returns a value greater than zero, then this condition will yield <code>true</code>             |
| <code>a &gt;= b</code> | if <code>a.compareTo(b)</code> returns a value equal to or greater than zero, then this condition will yield <code>true</code> |
| <code>a &lt; b</code>  | if <code>a.compareTo(b)</code> is less than zero, then this condition will yield <code>true</code>                             |
| <code>a &lt;= b</code> | if <code>a.compareTo(b)</code> is less than or equal to zero, then this condition will yield <code>true</code>                 |

## Comparison shopping

Remember the disco-friendly `LavaLamp` class I first defined in "[Feeling Groovy](#)," which also served as an example of migrating to the new JSR syntax in "[Groovy's growth spurt](#)"? I'm going to use this class again to demonstrate some nifty tricks with comparison operators.

In Listing 1, I've enhanced the `LavaLamp` class by implementing the normal-Java `equals()` method (and its partner in crime `hashCode`). Additionally, I've made `LavaLamp` implement the Java language's `Comparable` interface and created an implementation for the `compareTo()` method:

### Listing 1. Return of the `LavaLamp`!

```
package com.vanward.groovy

import org.apache.commons.lang.builder.CompareToBuilder
import org.apache.commons.lang.builder.EqualsBuilder
import org.apache.commons.lang.builder.HashCodeBuilder
import org.apache.commons.lang.builder.ToStringBuilder

class LavaLamp implements Comparable{
    String model
    String baseColor
    String liquidColor
    String lavaColor

    def String toString() {
        return new ToStringBuilder(this)
            .append(this.model)
            .append(this.baseColor)
            .append(this.liquidColor)
            .append(this.lavaColor)
            .toString()
    }

    def boolean equals(obj) {
        if (!(obj instanceof LavaLamp)) {
```

```

        return false
    }
    LavaLamp rhs = (LavaLamp) obj
    return new EqualsBuilder().
        append(this.model, rhs.model).
        append(this.baseColor, rhs.baseColor).
        append(this.liquidColor, rhs.liquidColor).
        append(this.lavaColor, rhs.lavaColor).
        isEqual()
    }

    def int hashCode() {
        return new HashCodeBuilder(17, 37).
            append(this.model).
            append(this.baseColor).
            append(this.liquidColor).
            append(this.lavaColor).
            toHashCode()
    }

    def int compareTo(obj) {
        LavaLamp lmp = (LavaLamp) obj
        return new CompareToBuilder().
            append(lmp.model, this.model).
            append(lmp.lavaColor, this.lavaColor).
            append(lmp.baseColor, this.baseColor).
            append(lmp.liquidColor, this.liquidColor).
            toComparison()
    }
}

```

**Note:** *Because I'm a big fan of re-use, I've relied heavily on the Jakarta's Commons Lang project for these implementations (even the `toString()` method!). If you're still coding your own `equals()` methods, you might want to take a minute to check out this library. (See [Resources](#).)*

In Listing 2, you can see the operator overloading I set up in Listing 1 in action. I've created five `LavaLamp` instances (yep, we're having a party!) and used Groovy's comparison operators to distinguish them from each other:

## Listing 2. Comparison operators in action

```

lamp1 = new LavaLamp(model:"1341", baseColor:"Black",
    liquidColor:"Clear", lavaColor:"Red")
lamp2 = new LavaLamp(model:"1341", baseColor:"Blue",
    liquidColor:"Clear", lavaColor:"Red")
lamp3 = new LavaLamp(model:"1341", baseColor:"Black",
    liquidColor:"Clear", lavaColor:"Blue")
lamp4 = new LavaLamp(model:"1342", baseColor:"Blue",
    liquidColor:"Clear", lavaColor:"DarkGreen")
lamp5 = new LavaLamp(model:"1342", baseColor:"Blue",
    liquidColor:"Clear", lavaColor:"DarkGreen")

println lamp1 <=> lamp2 // 1
println lamp1 <=> lamp3 // -1
println lamp1 < lamp3 // true
println lamp4 <=> lamp5 // 0

assert lamp4 == lamp5
assert lamp3 != lamp4

```

Note how `lamp4` and `lamp5` are the same and how the others vary subtly from each other. Because `lamp1`'s `baseColor` is `Black` and `lamp2`'s `baseColor` is `Blue`, `<=>` returned 1 (the *a* in black comes before the *u* in blue). Similarly, `lamp3`'s `lavaColor` is `Blue` as opposed to `lamp1`'s `Red`. Because the condition `lamp1 <=> lamp3` returned -1, the statement `lamp1 < lamp3` returns `true`. `!!lamp4` and `!!lamp5` are equal, therefore, `<=>` returns 0.

Also, do you see how `==` worked for object equality? Both `lamp4` and `lamp5` are the same. Of course, I could have used `assert lamp4.equals(lamp5)` to establish this, but `==` is so much quicker!

### I want my reference identity

Now, what if I really want to test object reference identity? Obviously, I can't use `==`, but Groovy does provide the `is()` method for just this sort of thing, as Listing 3 shows:

#### Listing 3. The `is()` works!

```
lamp6 = new LavaLamp(model:"1344", baseColor:"Black",
                    liquidColor:"Clear", lavaColor:"Purple")

lamp7 = lamp6
assert lamp7.is(lamp6)
```

As you can see in Listing 3, both `lamp6` and `lamp7` are the same reference, hence `is` returns `true`.

By the way, if you're getting dizzy, don't be surprised: Using overload operators turns the Groovy language almost backwards. But in a fun way, I think.

## Math-like operators

Groovy supports the following math-like operators for overloading:

**Table 3. Groovy's math-like operators**

| Operator                             | Method                     |
|--------------------------------------|----------------------------|
| <code>a + b</code>                   | <code>a.plus(b)</code>     |
| <code>a - b</code>                   | <code>a.minus(b)</code>    |
| <code>a * b</code>                   | <code>a.multiply(b)</code> |
| <code>a / b</code>                   | <code>a.divide(b)</code>   |
| <code>a++</code> or <code>++a</code> | <code>a.next()</code>      |
| <code>a--</code> or <code>--a</code> | <code>a.previous()</code>  |

```
a << b
```

```
a.leftShift(b)
```

You may have noticed that the `+` operator in Groovy is already overloaded in a few different areas, especially when it comes to collections. Have you ever wondered how this is possible, or at least wondered if you could do the same with your own classes? Well, now is the time to find out.

## Add up the operators

Remember the Song class from "[Stir some Groovy into your Java apps](#)"? Let's see what happens when I create a jukebox object to play those Songs. In Listing 4, I'll ignore the nitty-gritty details of actually playing MP3s and focus on adding and subtracting Songs from my JukeBox:

### Listing 4. Jukebox, baby

```
package com.vanward.groovy
import com.vanward.groovy.Song

class JukeBox {
    def songs

    JukeBox(){
        songs = []
    }

    def plus(song){
        this.songs << song
    }

    def minus(song){
        def val = this.songs.lastIndexOf(song)
        this.songs.remove(val)
    }

    def printPlayList(){
        songs.each{ song -> println "${song.getTitle()}" }
    }
}
```

By implementing the `plus()` and `minus()` methods, I've overloaded both `+` and `-` and can now use them in my scripts. Listing 5 demonstrates their behavior to add and subtract songs from a play list:

### Listing 5. Play that funky music

```
sng1 = new Song("SpanishEyes.mp3")
sng2 = new Song("RaceWithDevilSpanishHighway.mp3")
sng3 = new Song("Nena.mp3")
```

```
jbox = new JukeBox()
jbox + sng1
jbox + sng2
jbox + sng3

jbox.printPlayList() //prints Spanish Eyes, Race with the Devil., Nena

jbox - sng2

jbox.printPlayList() //prints Spanish Eyes, Nena
```

## Overloading overloaded overloaders

Continuing on with this, uh, *overloaded* theme, you may have noticed in Table 3 that one of the math-like operators that can be overridden is <<, which also happens to be overridden for Groovy collections. In the collections case, << has been overridden to act like the normal-Java add( ) method by appending the value to the end of a collection (this is also quite similar to Ruby). In Listing 6, you can see what happens when I mimic this behavior to allow users of my JukeBox to add Songs through the << operator, as well as the + operator:

### Listing 6. Left-shift that music

```
def leftShift(song){
    this.plus(song)
}
```

In Listing 6, I've implemented the leftShift method, which calls the plus method for adding Songs to my playlist. Listing 7 shows the << operator in action:

### Listing 7. The race is on

```
jbox << sng2 //re-adds Race with the Devil...
```

As you can see, Groovy's math-like overload operators can not only bear quite a *load*, but they can do it *apace*!

## Array-like operators

Groovy supports overloading the standard Java array access syntax [], as shown in Table 4:

**Table 4. Array operators**

| Operator | Method        |
|----------|---------------|
| a[b]     | a.getAt(b)    |
| a[b] = c | a.putAt(b, c) |

The array access syntax maps quite well to collections, so I update the JukeBox class to overload both cases in Listing 8:

### Listing 8. Music overload

```
def getAt(position){
  return songs[position]
}

def putAt(position, song){
  songs[position] = song
}
```

Now that I've implemented both `getAt` and `putAt`, I can use the `[]` syntax, as shown in Listing 9:

### Listing 9. Does it get any faster than this?

```
println jbox[0] //prints Spanish Eyes
jbox[0] = sng2  //placed Race w/the Devil in first slot
println jbox[0] //prints Race w/the Devil
```

## Groovier JDK methods

Once you've grasped the concepts of operator overloading and how it's done in Groovy, you can see that many everyday Java objects have *already* been enhanced by the writers of Groovy.

For example, the `Character` class supports `compareTo()`, as shown in Listing 10:

### Listing 10. Comparing chars

```
def a = Character.valueOf('a' as char)
def b = Character.valueOf('b' as char)
def c = Character.valueOf('c' as char)
def g = Character.valueOf('g' as char)

println a < b //prints true
println g < c //prints false
```

Likewise, the `StringBuffers` can be appended with the `<<` operator, as shown in Listing 11:

### Listing 11. Strings in the buff

```
def strbuf = new StringBuffer()
```



```
strbuf.append("Error message: ")
strbuf << "NullPointerException on line ..."

println strbuf.toString() //prints Error message: NullPointerException on line ...
```

And, finally, Listing 12 shows that Dates can be manipulated through + and -.

### Listing 12. Which day is it?

```
def today = new Date()

println today //prints Tue Oct 11 21:15:21 EDT 2005
println "tomorrow: " + (today + 1) //Wed Oct 12 21:15:21 EDT 2005
println "yesterday: " + (today - 1) //Mon Oct 10 21:15:21 EDT 2005
```

## Conclusion

As you've seen, operator ad hoc polymorphism, or operator overloading to the rest of us, can be quite powerful if carefully employed and documented. Beware of abusing this feature, however. If you decide to override an operator to do something unusual, be sure to clearly document your efforts. Enhancing your Groovy classes to support overloading is amazingly simple. Doing it with care and documenting your efforts is a fair trade-off for the wealth of handy shortcuts that result.

# Resources

## Learn

- ["Practically Groovy: Of MOPs and mini-languages"](#) (Andrew Glover, developerWorks, September 2005): If you think overloading operators is cool, check out the Meta Object Protocol!
- ["alt.lang.jre: Get to know Jython"](#) (Barry Feigenbaum, developerWorks, July 2004): Jython is another Java-based language that emphasizes flexibility with operator overloading.
- ["Interview with Guillaume Laforge"](#): Andrew talks with Groovy's project manager about the JSR, who's using Groovy, and Groovy's future.
- [The \*Practically Groovy\* series page](#): Tips and tricks for Groovy programmers.
- [The Java technology zone](#): Hundreds of articles about every aspect of Java programming.

## Get products and technologies

- [The Groovy project page](#): Download Groovy.
- [Commons Lang](#): A repository of extra methods for handling core class on the Java platform, including String manipulation methods.

## Discuss

- [developerWorks blogs](#): Get involved in the developerWorks community!

# About the authors

## Andrew Glover

Andrew Glover is a developer, author, speaker, and entrepreneur. He is the founder of the [easyb](#) Behavior-Driven Development (BDD) framework and is the co-author of three books: [Continuous Integration](#), [Groovy in Action](#), and [Java Testing Patterns](#). He teaches a wide variety of Groovy-, Grails-, and testing-related classes at [ThirstyHead.com](#). You can keep up with Andy at [thediscoblog.com](#) where he routinely blogs about software development.

---

## Scott Davis

Scott Davis is an internationally recognized author, speaker, and software developer. He is the founder of [ThirstyHead.com](#), a Groovy and Grails training company. His

books include [Groovy Recipes: Greasing the Wheels of Java](#), [GIS for Web Developers: Adding Where to Your Application](#), [The Google Maps API](#), and [JBoss At Work](#). He writes two ongoing article series for IBM developerWorks: [Mastering Grails](#) and [Practically Groovy](#).